

Rigid Bodies in MARS: Grid Interactions, Equations of Motion, and Energy Bookkeeping

October 2026

1 Notation and frames

A rigid body b has lab-frame position $\mathbf{r}_b$, orientation $\mathbf{R}_b \in SO(3)$ (columns are the body axes in the lab frame), linear momentum $\mathbf{p}_b$ (lab frame), and angular momentum $\mathbf{L}_b$ stored in the *body* frame. Its type supplies a mass m_b and principal moments $\mathbf{I}_b = (I_x, I_y, I_z)$; the body frame is assumed principal.

A grid G attached to a body has an origin $\mathbf{o}_G$ and a basis matrix $\mathbf{B}_G$ (columns are the voxel step vectors), both in the body frame. The voxel with integer index $\mathbf{n} = (n_x, n_y, n_z)$ sits at

$$\mathbf{x}_{\mathbf{n}} = \mathbf{R}_b (\mathbf{B}_G \mathbf{n} + \mathbf{o}_G) + \mathbf{r}_b = \mathbf{A}_G \mathbf{n} + \mathbf{O}_G, \quad \mathbf{A}_G \equiv \mathbf{R}_b \mathbf{B}_G, \quad \mathbf{O}_G \equiv \mathbf{R}_b \mathbf{o}_G + \mathbf{r}_b, \quad (1)$$

where $\mathbf{A}_G$ and $\mathbf{O}_G$ are the grid's lab-frame basis and origin. A lab point $\mathbf{x}$ maps to continuous index coordinates $\mathbf{s} = \mathbf{A}_G^{-1}(\mathbf{x} - \mathbf{O}_G)$. A grid owned by no body (an external PMF) uses $\mathbf{R} = \mathbf{1}$, $\mathbf{r} = \mathbf{0}$.

Grid values are interpolated in index space (linear or cubic), giving $\tilde{u}(\mathbf{s})$ and $\nabla_{\mathbf{s}} \tilde{u}$. By the chain rule the lab-frame gradient is

$$\nabla_{\mathbf{x}} u = \mathbf{A}_G^{-\top} \nabla_{\mathbf{s}} \tilde{u}. \quad (2)$$

Every grid enters with a scale factor λ (`GridTerm::scale`, config `gridFileScale`).

Reference point. All torques on body b are taken about $\mathbf{r}_b$, because the integrators (Section 5) apply $\mathbf{R}_b^\top \boldsymbol{\tau}_b$ to the body-frame angular momentum. If a torque $\boldsymbol{\tau}(\mathbf{c})$ is computed about a point $\mathbf{c}$ for forces with sum $\mathbf{F}$, then

$$\boldsymbol{\tau}(\mathbf{r}_b) = \boldsymbol{\tau}(\mathbf{c}) + (\mathbf{c} - \mathbf{r}_b) \times \mathbf{F}. \quad (3)$$

2 Grid–grid interaction between two bodies

2.1 Energy

Body A carries a *density* grid ρ and body B a *potential* grid u . Grid pairs are matched by key name: a density grid of type t_i pairs with a potential grid of type t_j with the same key, for $t_i \leq t_j$ (the legacy one-direction rule, kept on purpose; `RigidBodyForcePairList`). The interaction energy is the overlap integral, discretized over ρ 's voxels:

$$E_{AB} = \lambda \int \rho(\mathbf{x}) u(\mathbf{x}) d^3x \approx \lambda \sum_{\mathbf{n}} \rho_{\mathbf{n}} \tilde{u}(\mathbf{s}_{\mathbf{n}}), \quad \mathbf{s}_{\mathbf{n}} = \mathbf{A}_u^{-1}(\mathbf{x}_{\mathbf{n}} - \mathbf{O}_u), \quad (4)$$

with $\mathbf{x}_n$ from Eq. (1) for grid ρ on body A . In the kernel the index map is $\mathbf{s}_n = \mathbf{A}_u^{-1}(\mathbf{A}_\rho \mathbf{n} + \mathbf{\Delta})$ with $\mathbf{\Delta} = \mathbf{O}_\rho - \mathbf{O}_u$ (`origin_offset`).

Note. `v1` stored $\sum_n \rho_n$ as the energy, which does not depend on the bodies' relative pose. MARS uses Eq. (4), the only energy whose negative gradient is the force both codes compute.

2.2 Force and torque on A

Each voxel is a point load. Its force is

$$\mathbf{f}_n = -\lambda \rho_n \nabla_{\mathbf{x}} u(\mathbf{x}_n) = -\lambda \rho_n \mathbf{A}_u^{-\top} \nabla_{\mathbf{s}} \tilde{u}(\mathbf{s}_n), \quad (5)$$

and

$$\begin{aligned} \mathbf{F}_A &= \sum_n \mathbf{f}_n, & \boldsymbol{\tau}_A(\mathbf{O}_\rho) &= \sum_n (\mathbf{A}_\rho \mathbf{n}) \times \mathbf{f}_n, \\ \boldsymbol{\tau}_A(\mathbf{r}_A) &= \boldsymbol{\tau}_A(\mathbf{O}_\rho) + (\mathbf{O}_\rho - \mathbf{r}_A) \times \mathbf{F}_A. \end{aligned} \quad (6)$$

The kernel accumulates $\boldsymbol{\tau}_A$ about $\mathbf{O}_\rho$ (the lever arm $\mathbf{A}_\rho \mathbf{n}$ is the voxel's offset from the density grid's origin), then applies the shift (3) once per block with $\mathbf{O}_\rho - \mathbf{r}_A = \mathbf{R}_A \mathbf{o}_\rho$ (`rho_shift`).

2.3 Reaction on B

E_{AB} is unchanged when both bodies are translated or rotated together, so the discretized sum conserves total linear and angular momentum exactly. The force and torque on B therefore follow in closed form, with no second grid pass:

$$\mathbf{F}_B = -\mathbf{F}_A, \quad \boldsymbol{\tau}_B(\mathbf{r}_B) = -\boldsymbol{\tau}_A(\mathbf{r}_A) + (\mathbf{r}_B - \mathbf{r}_A) \times \mathbf{F}_A. \quad (7)$$

The kernel uses the equivalent form $\boldsymbol{\tau}_B(\mathbf{O}_u) = -[\boldsymbol{\tau}_A(\mathbf{O}_\rho) + \mathbf{\Delta} \times \mathbf{F}_A]$, shifted to $\mathbf{r}_B$ with $\mathbf{O}_u - \mathbf{r}_B = \mathbf{R}_B \mathbf{o}_u$ (`u_shift`). Expanding $\mathbf{\Delta}$ shows it equals Eq. (7). It needs no body positions, and it uses the same (unwrapped) $\mathbf{\Delta}$ as the force.

2.4 External PMF grids

A density grid can also be matched against one of its own type's PMF grids. The PMF is fixed in the lab frame ($\mathbf{A}_u = \mathbf{B}_u$, $\mathbf{O}_u = \mathbf{o}_u$). There is no second body, so Eq. (7) does not apply and A keeps all of E .

2.5 Update period

A pair may be evaluated only every P steps (`rigidBodyGridGridPeriod`). For the time-averaged impulse to be the same as evaluating every step, the force and torque applied on an evaluation step should be multiplied by P , as `v1` does.

2.6 Kernels

`RBGridCullKernel` keeps (instance pair, grid pair) candidates within the cutoff, or always keeps them for a PMF, and precomputes $\mathbf{A}_\rho$, $\mathbf{A}_u^{-1}$ and $\mathbf{\Delta}$. `RBGridPrefixSumKernel` assigns block ranges. `RBGridBatchedForceKernel` splits each item's ρ voxels over its blocks, evaluates `grid.grid_voxel_force_torque`, block-reduces $(\mathbf{F}, E)$ and $\boldsymbol{\tau}$, and atomically adds the results into the bodies (`RigidBodyGridBatch.h`, `GridGridKernels.h`).

3 Particles in a body's potential grid

3.1 Selection

A particle type samples body potential grid u only if its `particle` block lists the grid's key with `rigidBodyPotential`. At dispatch time one candidate is made per (body instance, potential grid) for which at least one particle type matches, along with a per-type flag row; the kernel skips particles whose type flag is 0. This is `vl`'s `partRigidBodyGrid` rule. A type mask is used instead of per-grid particle-index lists because it stays valid when particles are reordered.

3.2 Force, torque, energy

For a selected particle k at $\mathbf{x}_k$, with $\mathbf{s}_k = \mathbf{A}_u^{-1}(\mathbf{x}_k - \mathbf{O}_u)$:

$$U_k = \lambda \tilde{u}(\mathbf{s}_k), \quad \mathbf{f}_k = -\lambda \mathbf{A}_u^{-\top} \nabla_{\mathbf{s}} \tilde{u}(\mathbf{s}_k), \quad (8)$$

$$\mathbf{F}_b = -\sum_k \mathbf{f}_k, \quad \boldsymbol{\tau}_b(\mathbf{O}_u) = \sum_k (\mathbf{x}_k - \mathbf{O}_u) \times (-\mathbf{f}_k), \quad (9)$$

and $\boldsymbol{\tau}_b(\mathbf{r}_b) = \boldsymbol{\tau}_b(\mathbf{O}_u) + (\mathbf{O}_u - \mathbf{r}_b) \times \mathbf{F}_b$ by Eq. (3), with $\mathbf{O}_u - \mathbf{r}_b = \mathbf{R}_b \mathbf{o}_u$ (`grid_shift`). Each particle receives $\mathbf{f}_k$ by atomic scatter, because several candidates can act on the same particle. The body's totals are block-reduced, then added atomically (`RBParticleGridForceKernel`, `RigidBodyParticleGridBatch.h`).

4 Attached particles

An attached particle k is an ordinary particle held at a fixed body-frame offset $\mathbf{d}_k$:

$$\mathbf{x}_k = \mathbf{R}_b \mathbf{d}_k + \mathbf{r}_b. \quad (10)$$

Its position is rewritten from the body each step (`RBSyncAttachedPositionsKernel`) and is not integrated. Every force path (pairwise, bonded, PMF, body grids) acts on it as on any other particle. The total force on it is then transferred to the body (`RBReduceAttachedForcesKernel`):

$$\mathbf{F}_b += \sum_k \mathbf{f}_k, \quad \boldsymbol{\tau}_b(\mathbf{r}_b) += \sum_k (\mathbf{R}_b \mathbf{d}_k) \times \mathbf{f}_k. \quad (11)$$

The lever arm is measured from $\mathbf{r}_b$, so $\mathbf{r}_b$ must be the point the attached template is built about (`referencePoint`). The particle's own force is left in place; nothing reads it afterwards, and the force output then still shows what acted on the particle.

5 Equations of motion

Let $\mathbf{F}_b$ and $\boldsymbol{\tau}_b$ (about $\mathbf{r}_b$, lab frame) be the totals from Sections 2–4, plus any host-set external load. The body-frame torque is $\mathbf{R}_b^\top \boldsymbol{\tau}_b$.

Langevin / DLM (`RBDLM.h`). Langevin forces are added first (`RBLangevinForceKernel`). In the body frame, per axis α ,

$$f_\alpha = \sqrt{\frac{2k_B T m \gamma_\alpha}{\Delta t}} \xi_\alpha - \gamma_\alpha \left(\mathbf{R}^\top \mathbf{p} \right)_\alpha, \quad \tau_\alpha = \sqrt{\frac{2k_B T I_\alpha \Gamma_\alpha}{\Delta t}} \eta_\alpha - \Gamma_\alpha L_\alpha, \quad (12)$$

with $\xi, \eta \sim \mathcal{N}(0, 1)$ and translational and rotational damping γ, Γ . These are rotated to the lab frame and added to $\mathbf{F}_b, \boldsymbol{\tau}_b$. The dissipative term carries legacy unit factors (`langevin_damping_unit`, `langevin_damp_scale`). The step is split DLM (Dullweber–Leimkuhler–McLachlan): half kick $\mathbf{p} += \frac{\Delta t}{2} \mathbf{F}$, $\mathbf{L} += \frac{\Delta t}{2} \mathbf{R}^\top \boldsymbol{\tau}$; drift $\mathbf{r} += \Delta t \mathbf{p}/m$; free rotation by the symmetric splitting $x(\frac{\Delta t}{2}) y(\frac{\Delta t}{2}) z(\Delta t) y(\frac{\Delta t}{2}) x(\frac{\Delta t}{2})$ each factor being the exact rotation about one body axis with rate L_α/I_α ; then the second half kick after the next force evaluation.

Brownian (RBBD.h). Overdamped, with no momenta. With mobilities $\mu_\alpha = 1/(\gamma_\alpha m)$ and $\mu_\alpha^r = 1/(\Gamma_\alpha I_\alpha)$ in the body frame,

$$\begin{aligned}\delta \mathbf{r}_{\text{body}} &= \mu \odot \mathbf{R}^\top \mathbf{F} \Delta t + \sqrt{2k_B T \mu \Delta t} \odot \boldsymbol{\xi}, \\ \delta \boldsymbol{\theta} &= \mu^r \odot \mathbf{R}^\top \boldsymbol{\tau} \Delta t + \sqrt{2k_B T \mu^r \Delta t} \odot \boldsymbol{\eta},\end{aligned}\tag{13}$$

then $\mathbf{r} += \mathbf{R} \delta \mathbf{r}_{\text{body}}$ and $\mathbf{R} \leftarrow \mathbf{R} \mathcal{R}(\delta \boldsymbol{\theta})$, where $\mathcal{R}$ is the symmetric $zyxz$ splitting.

Units. Lengths are Å, time ns, mass amu, energy kcal/mol. Force times time becomes momentum through `impulse_to_momentum` ($= 4.1868 \times 10^4$), and $\mathbf{p}/m$ becomes Å/ns through `velocity_scale` ($= 10^4$). The stored momentum is therefore in units where $\mathbf{p}/m$ is measured in 10^4 Å/ns $= 1$ km/s.

6 Energy

6.1 Potential energy: shares

Every interaction energy is divided once among its partners, so the shares add up to the total with nothing counted twice. Each particle stores its share in the fourth component of its force (`ForceEnergy.t`); each body stores its share in `rb.force[b].t`.

Interaction	Particle share	Body share
Nonbonded pair, bond	$\frac{1}{2}$ each	—
Angle / dihedral	$\frac{1}{3} / \frac{1}{4}$ each	—
Restraint, particle PMF	all	—
Particle in body potential grid	$\frac{1}{2}$	$\frac{1}{2}$
Body–body grid pair	—	$\frac{1}{2}$ each
Body density in external PMF	—	all

The split is by partner count, not by mass. Potential energy belongs to the interaction rather than to either partner, and mass only determines how *kinetic* energy is shared.

Writing $U_b^{\text{grid}} = \text{rb.force}[b].t$, the system potential energy is

$$U_{\text{total}} = \sum_{k \in \text{particles}} U_k + \sum_b U_b^{\text{grid}}.\tag{14}$$

A body whose type has no grids has $U_b^{\text{grid}} = 0$.

Attached particles are ordinary particles, so their energy is already in $\sum_k U_k$. It is also reported for each body as

$$U^{\text{att}} = \sum_{k \in \text{attached}} U_k,\tag{15}$$

because for a body that is just a collection of particles this, not U^{grid} , is the body’s interaction energy. Which term is meaningful depends on the model, so both are reported and neither is assumed.

6.2 Kinetic energy

$$K_t = \frac{\mathbf{p}^2}{2m}, \quad K_r = \sum_{\alpha} \frac{L_{\alpha}^2}{2I_{\alpha}} \quad (\text{body frame}). \quad (16)$$

In stored units $\mathbf{p}^2/m$ is in $\text{amu}(\text{km/s})^2 = 1 \text{ kJ/mol}$, so

$$K [k_B T] = \frac{K_{\text{raw}}}{4.1868 k_B T [\text{kcal/mol}]}, \quad 4.1868 = \text{SQRT_CAL_TO_JOULE}^2, \quad (17)$$

which reproduces v1's factor $2.388458509 \times 10^{-1} = 1/4.1868$ and matches the particle kinetic energy.

6.3 Reduction and output

`RBEnergyReduceKernel` (`RBOperation/RBEnergyKernels.h`) has thread i handle body i and attached particle i . It block-reduces one `RB_Energy` = $(K_t, K_r, U^{\text{grid}}, U^{\text{att}})$ and atomically adds it into a single device value, so one 16-byte copy reaches the host on each energy-output step. `rb_energy.dat`:

```
Kinetic Energy <Kt+Kr> (kT)
  Translational <Kt>   Rotational <Kr> (kT)
Potential Energy <Ugrid+Uatt> (kcal/mol)
  Grid <Ugrid>   Attached <Uatt> (kcal/mol)
```

The first and third lines keep the v1 format.

7 Implementation status

The code departs from the equations above in these places:

1. **Update period.** Forces and torques from pairs with $P > 1$ are not multiplied by P .
2. **SMD scale slope.** `gridFileScaleSlope` is not applied to body potential grids.
3. **Periodicity.** Grid–grid offsets are not wrapped; periodic body–body pairs are future work.